M.Tech Program

Advanced Industry Integrated Programs

Jointly offered by University and LTI MindTree

Intelligent Cache Optimization for AI Agents

Review 3

DEV S SHAH
VIT CHENNAI
T9991

Knowledge partner

LTM

Implementation Partner

L&T EduTech

Introduction

Problem

- Repeated LLM queries cause **high cost, latency, and token wastage**
- Scalability challenges due to growing API usage and rate limits

Solution: Intelligent Caching

- Exact Match Cache : Redis
- Semantic Similarity Cache : Pgvector
- Context-Aware Retrieval : Agent/Tool support.
- **Intelligent Policy Engine:** Cost/Frequency/Recency based eviction/caching.

Objective

- **Tiered Cache Middleware:** Multi-stage interceptor for LLM/Agent calls.
- **Intelligent Storage Layer:**
  - *Exact:* Redis for deterministic lookups (low latency).
  - *Semantic:* Pgvector for context/meaning matches (prevents redundant compute).
- **Policy-Driven Optimization:** Intelligent scorer evaluates *if* result worth caching (balances cost vs. memory).
- **Observability Suite:** Streamlit dashboard tracking live hit-rates, latency, and cost savings.

Module Name : ICO - cache

- Core Application layer
- Cache Implementation Layer
- Intelligence layer
- LLM provider layer
- Support Components
- Dashboard and Monitoring
- Benchmarking FrameWork

Problem Statement (Why ico-cache Graph is better)

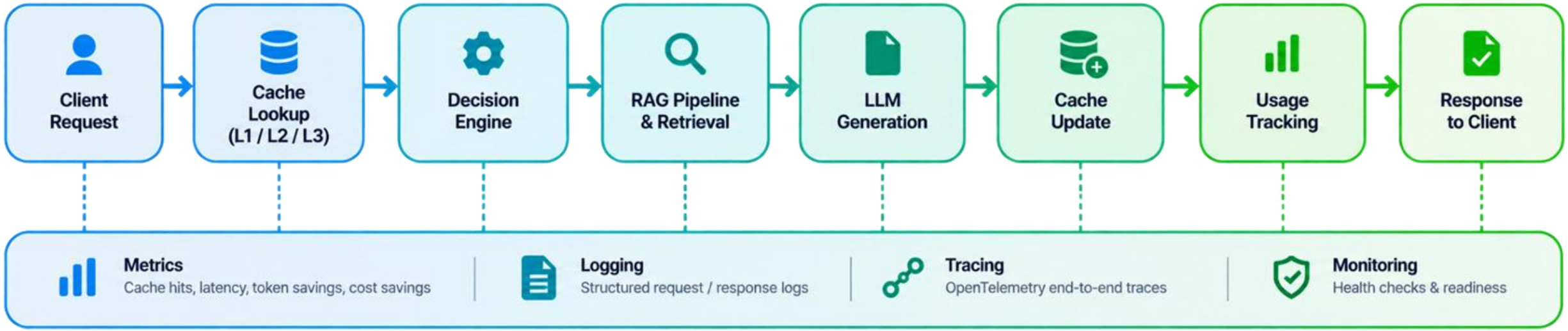
1. Wasted cost & time — repeated or reworded queries re-run through the LLM, paying full token cost and 2–10s latency every time.

2. Unsafe caching — naïve semantic caches return false hits, causing cross-entity, cross-tenant, and cross-quarter answer bleed.

3. No production-ready solution — existing caches are either exact-match only, infra-heavy, platform-fragile, or not context-aware.

4. No visibility into savings — teams can't measure how many LLM calls, tokens, and dollars a cache actually eliminates.

Methodology



Approach/Algorithms/Tools

Area	Approach / Algorithm	Tool Used
L1 Exact cache	SHA-256 hash of normalized query + model + prompt_version + metadata	Redis / SQLite
L2/L3 Semantic, Context aware cache	FastEmbed vectors (384-dim) + cosine similarity ≥0.85/0.90, dual-vector for context	Qdrant / LanceDB, BAAI/bge-small-en-v1.5
Anti-false-hit guard	Metadata hard-gate on tenant/entity/quarter/model; mismatch = block	metadata_guard.py + decision_engine.py
Document ingestion	Content-sniffing + chunking + OCR fallback	PyMuPDF, beautifulsoup4, tree-sitter, Tesseract
Concurrency + tenancy	Single-flight coalescing, tenant_id isolation	asyncio, Redis, Qdrant collections
Observability + QA	Metrics/tracing/cost accounting, deterministic benchmarks	Prometheus, OpenTelemetry, pytest, benchmark.py

Findings and Results

Measured per block (fresh cache state):

Block	Requests	Cache hits	Winning tiers	Tokens saved
L0a deterministic	30	30 (100%)	L0a ×30	377
L1 exact match	30	16 (53%)	L1 ×14, L2 ×2	2,270
L2 semantic paraphrase	30	24 (80%)	L2 ×13, L1 ×11	3,491
L3 context-aware dialogue	50	16 (32%)	L3 ×16	2,296
L4/L5 bypass retrieval	30	0*	internal L4/L5	0
Mixed verification	30	30 (100%)	L0a ×10, L1 ×18, L3 ×2	2,902
Total	**200**	**116 (58.0%)**		**11,336**

Conclusion… Calibri body -36

Achievements

- Developed an end-to-end system that converts SEC filings into a queryable financial knowledge graph
- Delivered accurate answers with source citations and zero hallucinated financial figures
- Ensured strict company isolation — no cross-company data mixing in results

Future Work

- Build graph segments on-the-fly during query time instead of pre-loading all filings
- Auto-process new filings daily so the system stays up-to-date without manual runs
- Reduce storage and startup time by loading only the data each query needs

Refrenences

